

TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE ENGINEERING AND AUTOMATICS

REPORT

on laboratory work no. 3
Course: **Operating Systems**

Topic: The Command Line: Shell, Pipes and Text Filters

Done by: st. gr. FAF-241
Vitalie Vasilean

Checked by:
univ. assist. / lect. Patricia Reiþman

Chişinău, 2026

Purpose of the work

To navigate the bash shell efficiently, predict shell expansions and command executions, redirect input/output streams, and answer real questions about data using small text filters joined by pipelines.

1 Part 1. Move fast, find help

1.1 Outputs and Work Done

```
vitalie@LinuxLabs:/$ type cd; type ls; type echo; which ls
cd is a shell builtin
ls is aliased to 'ls --color=auto'
echo is a shell builtin
/usr/bin/ls

vitalie@LinuxLabs:/$ echo $PATH | tr ':' '\n'
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
/usr/local/games
/snap/bin

vitalie@LinuxLabs:/$ nosuchprog; echo "exit code: $?"
nosuchprog: command not found
exit code: 127
```

1.2 Observe

1. **Built-ins vs Externals:** `cd` and `echo` are shell built-ins, while `ls` is an external command. `cd` MUST be a built-in because a child process cannot change the working directory of its parent process (the shell).
2. **PATH variables:** There are 9 directories listed in my `PATH`. The `ls` executable is located in `/usr/bin/ls`.
3. **Exit codes:** A command that is not found returns the standard exit code 127.
4. **Manual mapping:** According to the `man ls` pages, the option to sort by modification time is `-t`.

2 Part 2. What the shell does to your line

2.1 Predictions and Outputs

Command	My prediction	Real output	Explanation (if different)
echo *.c	a.c b.c	a.c b.c	Shell expands the wildcard to matching files.
echo file?.txt	file1.txt file2.txt	file1.txt file2.txt	'?' matches exactly one character.
echo [a-b]*	a.c b.c	a.c alfa beta.txt b.c	'*' matches remainder of strings starting with a or b.
echo *.xyz	error / nothing	*.xyz	If no match exists, the literal string is passed.
ls alfa beta.txt	'alfa beta.txt'	cannot access 'alfa'...	Unescaped spaces act as argument separators.
ls "alfa beta.txt"	'alfa beta.txt'	'alfa beta.txt'	Quotes treat the string as a single argument.
echo \$HOME "\$HOME"	/home/vitalie twice	/home/vitalie /home/vitalie	Double quotes evaluate variables.
echo '\$HOME' \ \$HOME	literal text	\$HOME \$HOME	Single quotes and backslashes escape variables.
echo {1..5}; echo \$((7*6))	1 2 3 4 5 and 42	1 2 3 4 5 42	Brace expansion and arithmetic evaluation.

2.2 Observe

1. See the table above for predictions and explanations.
2. **Globbing:** The shell (Bash) itself produced the file names. It matches the wildcard `*.c` to actual files in the directory and replaces the wildcard with the file names before passing them as arguments to `echo`.
3. **Fixing ls:** The command failed because the space separated it into two distinct arguments (`alfa` and `beta.txt`). The two ways to fix it are using quotes (`ls "alfa beta.txt"`) or escaping the space (`ls alfa\ beta.txt`).
4. **Quotes:** Double quotes (`" "`) allow variable expansion (like `$HOME`), whereas single quotes (`' '`) treat everything literally. Modern `ls` wraps output containing spaces in single quotes to clearly denote it is a single file and prevent user ambiguity.

3 Part 3. Redirection, chaining and jobs

3.1 Outputs and Work Done

```
vitalie@LinuxLabs:~/os-lab3$ ls /etc/nope > out.txt 2> err.txt
vitalie@LinuxLabs:~/os-lab3$ wc -l out.txt; cat err.txt
0 out.txt
ls: cannot access '/etc/nope': No such file or directory

vitalie@LinuxLabs:~/os-lab3$ wc -l < access.log
500

vitalie@LinuxLabs:~/os-lab3$ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2

vitalie@LinuxLabs:~/os-lab3$ ls /etc/nope > all.txt 2>&1
vitalie@LinuxLabs:~/os-lab3$ ls /etc/nope 2>&1 > all.txt
ls: cannot access '/etc/nope': No such file or directory
```

3.2 Observe

1. **Redirection targets:** The `err.txt` file received the error message (stderr redirected via `2>`), and `out.txt` received the standard listing output (which was empty).
2. **Order of redirection:** Shell reads redirections left to right. `> all.txt 2>&1` sends stdout to the file, then redirects stderr to wherever stdout is currently pointing (the file). Reversing it to `2>&1 > all.txt` redirects stderr to the screen (stdout's original destination) first, and then redirects stdout to the file, leaving the error on the screen.
3. **Standard Input:** `wc -l < access.log` redirects the file's text stream via standard input, meaning `wc` receives text but doesn't know the file name. Supplying it as an argument (`wc -l access.log`) passes the file metadata, allowing `wc` to print the name.
4. **Operators:**
 - `;` runs the next command sequentially regardless of the first command's success or failure.

- `&&` (AND) runs the next command *only* if the previous command succeeded (exit code 0).
- `||` (OR) runs the next command *only* if the previous command failed (non-zero exit code).

4 Part 4. Answer questions with one-liners

4.1 Outputs and Work Done

```
vitalie@LinuxLabs:~/os-lab3$ bash oneliners.sh
100
    125 /page0
    125 /page1
    125 /page2
    125 /page3
    125 /page3
    125 /page2
    125 /page1
    125 /page0
    15 user5
Total requests by user3: 72 | Failed: 15
10:10 user0
10:15 user5
10:20 user3
    40 /usr/sbin/nologin
     6 /bin/false
     2 /bin/bash
     1 /bin/sync
Bonus - Failed at exact 10 minutes:
50
```

4.2 Observe

Q8. `ls -l /etc | wc -l` returns exactly one more line than `ls /etc | wc -l` because the `-l` (long list) format outputs a "total" summary block size on the very first line (e.g., `total 128`) before listing the files. The standard `ls` does not print this summary line.

5 Part 5. Your first script

5.1 Outputs and Work Done

```
vitalie@LinuxLabs:~/os-lab3$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures

vitalie@LinuxLabs:~/os-lab3$ ./report.sh nope.log; echo "exit code: $?"
no such file: nope.log
exit code: 1
```

```
vitalie@LinuxLabs:~/os-lab3$ cat ~/os-lab3/report.log
500 requests, 100 failed
user0: 14 failures
...
```

5.2 Observe

1. **Shebang:** The first line `#!/bin/bash` (the shebang) instructs the operating system to use the Bash interpreter to execute the script. Running the script before making it executable with `chmod +x` results in a "Permission denied" error.
2. **Pathing:** You must type `./report.sh` to explicitly tell the shell to look in the current directory (`.`). Otherwise, the shell only searches the directories listed in the `$PATH` variable and will return "command not found".
3. **Error Handling:** The script uses `>&2` to redirect the error message specifically to the standard error stream (`stderr`) rather than normal output. It ends with `exit 1` to signal to the OS and parent processes that the script terminated in a failure state.

Conclusions

Through this laboratory work, I successfully set up and interacted with the Linux Bash shell. I learned how the shell interprets command-line arguments via globbing and expansion prior to execution. I practically applied input and output redirection to manage command outputs and errors, and effectively chained commands using logical operators. Furthermore, by constructing shell pipelines utilizing utilities like `grep`, `cut`, `sort`, and `uniq`, I was able to perform efficient data extraction and analysis on log files. Finally, converting these commands into a parameterized, executable Bash script solidified my understanding of shell scripting, exit codes, and automated cron task scheduling.